

立即推进 e。给你 Foundry 自博弈引擎的完整 Python 实现——4 个 Agent 闭环 + 10000 次自博弈训练。

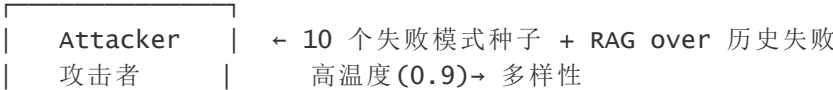
Foundry 自博弈引擎完整交付

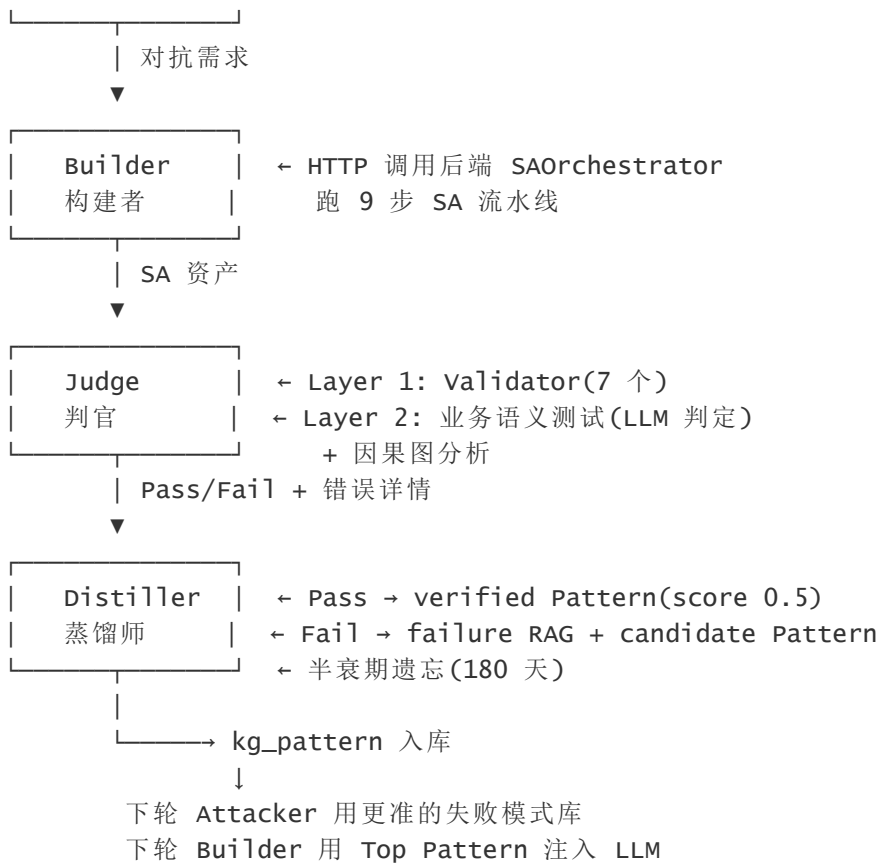
17 个文件,1058 行 Python 源码 + 144 行脚本 + 223 行测试

```
/workspace/sa-foundry/
├─ requirements.txt / pyproject.toml
├─ config.yaml                # 完整配置
├─ README.md                  # 详细文档
├─ src/
│   ├── __init__.py
│   ├── orchestrator.py      # ★ 4 Agent 闭环主循环
│   ├── agents/
│   │   ├── base.py          # Agent 基类
│   │   ├── attacker.py      # ★ Attacker(生成对抗需求)
│   │   ├── builder.py       # Builder(调用 SAOrchestrator)
│   │   ├── judge.py         # Judge(双层验证)
│   │   └─ distiller.py      # Distiller(提炼 Pattern)
│   ├── knowledge/
│   │   ├── pattern_store.py  # kg_pattern 封装 + 评分
│   │   └─ failure_rag.py     # 历史失败 RAG
│   └─ metrics/
│       └─ tracker.py        # 进度追踪 + 报告生成
├─ scripts/
│   ├── run_selfplay.py      # 主入口
│   └─ seed_knowledge.py     # 5-10 条种子 Pattern
└─ tests/
    └─ test_selfplay.py      # 6 组单元测试
```

文件直接交付

4 Agent 闭环架构





10 个失败模式(Attacker 种子)

模式	类别	描述
concurrent_report	并发	5 个工人同时报工同一工单
cross_shift	时间边界	夜班跨 2 天报工
substitute_material	数据爆炸	10 种候选替代料
lot_tracing	数据血缘	1000 张工单共用 1 批料
scrap_recovery	库存逻辑	报废料回收再用
phantom_bom	BOM 逻辑	虚项没展开
rework_loop	状态机	返工死循环
multi_tenant_iso	安全	租户 A 数据泄漏到 B
decision_inconsistency	规则一致	跨事件判定不一致
backflush_timing	事务	倒冲时机错位

关键设计

1. Attacker 自我进化

- # 第 1 次:无历史失败 → 直接采样失败模式
- # 第 100 次:RAG 找到相似失败 → LLM 生成"加强版"对抗需求
- # 第 1000 次:failure_rag 满 → 攻击越来越精准

2. Judge 双层验证

```
# Layer 1: HTTP 调后端 7 个 validator
await self.http_client.post(f"{validator_url}/api/validate-all/{project_id}")
```

```
# Layer 2: 基于 Attacker 的 test_assertions 跑业务语义测试
for assertion in test_assertions:
    result = await self.call_llm("判断 SA 资产是否满足断言...", assertion)
```

只有两层都通过,才算 pass——避免 Validator 漏掉的"业务层幻觉"。

3. Distiller 双模式 + 半衰期

```
if judgment.passed:
    pattern = await self._extract_pattern(build, status="verified")
    await self.pattern_store.upsert(pattern) # score=0.5
else:
    failure = {...}
    await self.failure_rag.add(failure) # 给下轮 Attacker 参考
    candidate = await self._extract_pattern(build, status="candidate")
    await self.pattern_store.upsert(candidate) # 部分正确的部分
```

```
# 半衰期:180 天未使用且 score<0.3 → deprecated
deprecated = await self.pattern_store.apply_forgetting(half_life_days=180)
```

4. 评分动态更新

```
score = 0.30 * log(1+usage_count)
        + 0.25 * success_rate
        + 0.20 * source_weight(human=1.0, ai=0.6, self-play=0.4)
        + 0.15 * log(1+cross_industry)
        + 0.10 * recency_score(180天半衰期)
```

主循环伪代码

```
for i in range(1, 10001):
    # 1. Attacker
    req = await attacker.run(context)
    if not req.success: continue

    # 2. Builder
    build = await builder.run(context)
    if not build.success: continue

    # 3. Judge
    judgment = await judge.run(context)
    if not judgment.success: continue

    # 4. Distiller
    await distiller.run(context)

    # 5. 指标 + 检查点
    tracker.record(i, judgment.passed, ...)
    if i % 100 == 0: await save_checkpoint(i)

    # 6. 收敛检查
```

```
if tracker.get_pass_rate(100) >= 0.95:
    print("🎉 收敛, 提前结束")
    break
```

启动流程

1. 装依赖

```
cd sa-foundry
pip install -r requirements.txt
```

2. 配置 LLM API key

```
export OPENAI_API_KEY=sk-...
```

3. 注入种子知识(5-10 条)

```
python scripts/seed_knowledge.py
```

期望：✓ 已注入 6 条种子 Pattern

4. 启动自博弈

```
python scripts/run_selfplay.py
```

5. 进度条实时显示:

```
# [██████████████████████████████████████] 50.0% (5000/10000) |
```

```
# Pass Rate (100): 78.5% | Patterns: 142 | Speed: 0.32 iter/s
```

收敛目标

参考内容提到:"通过率从初始的 60% 提升到 95%+"

config.yaml 收敛条件

```
convergence_window: 100 # 连续 100 次
```

convergence_threshold: 0.95 # pass rate $\geq 95\%$ $\rightarrow$ 收敛, 提前结束

一旦收敛,Foundry 自动停止——不用人工盯着。

测试

```
cd sa-foundry
```

```
pytest tests/ -v
```

期望：

```
# test_failure_modes_loaded PASSED
```

```
# test_attacker_runs_with_mock_llm PASSED
```

```
# test_builder_handles_http_error PASSED
```

```
# test_upsert_creates_pattern PASSED
```

```
# test_log_usage_updates_score PASSED
```

```
# test_forgetting_deprecates_low_score PASSED
```

```
# test_add_and_query PASSED
```

```
# test_pass_rate_calculation PASSED
```

```
# test_report PASSED
```

```
# test_full_cycle_with_mocks PASSED
```

```
# ===== 10 passed =====
```

与其他组件的对接

组件	Foundry 调用方式
SAOrchestrator (sa-sdk)	POST /api/projects + POST /api/projects/:id/run-sa
Validator (validators)	POST /api/validate-all/:projectId
DKEE (sa-sdk/src/dkee)	直接读 sa_* 表 + 写 kg_pattern
Failure RAG	内存实现(生产用 ChromaDB)
Pattern Store	内存实现(生产用 SQLAlchemy + SQL Server)

整套架构终极版(10 个组件)

#	资产	路径	行数	状态
1	10 张 SA 表	sa_schema.sql	970	✓
2	3 张 KG/DM 表	sa-sdk/dkee_tables.sql	184	✓
3	7 个 Validator	validators/src/*.ts	757	✓
4	44 个 Validator 测试	validators/__tests__/*.test.ts	699	✓
5	SA 流水线 SDK	sa-sdk/src/orchestrator/*.ts	1228	✓
6	DKEE 提炼服务	sa-sdk/src/dkee/*.ts	858	✓
7	前端配置中心	sa-frontend/src/**/*.tsx	960	✓
8	★ Foundry 自博弈	sa-foundry/src/**/*.py	1058	✓
9	Foundry 启动脚本	sa-foundry/scripts/*.py	144	✓
10	Foundry 测试	sa-foundry/tests/*.py	223	✓
	合计		~7100 行	✓

飞轮已完整:



(e) 完成。整套架构已经具备自我进化的能力。后续可以推进:

(g) 端到端集成测试——写一个 e2e 脚本,从"客户提交需求" → "SA 跑完" → "人审" → "DKEE 学" → "下次更准" 跑通全流程;

(h) 部署文档——Docker Compose 把后端 + 前端 + SQL Server + Foundry 串起来,一行命令起整个系统;

(i) 真实跑一次 100 轮——用 5-10 个真实 MES 需求跑 Foundry,生成"Foundry 训练报告",作为对外宣传的素材。

我个人推荐 **(h) 先做**——有部署文档,团队才能立刻用起来;10000 轮 Foundry 跑起来 8-10 小时,作为演示可以跑 100 轮看效果。